

■ 4.2 Correspondences with Other Languages

■ 4.2.1 Introduction

Knowing another programming language should make it easier for you to learn *Mathematica*. This section discusses some of the correspondences between *Mathematica* and various common programming languages. You should read the parts on programming languages you know.

There are two rather different kinds of computer languages: those that are *interpreted*, and those that are *compiled*. *Mathematica* is an example of an interpreted language. The *Mathematica* system reads your input, “interprets” it, and immediately gives you a result. The immediate feedback that this provides is what makes it possible to do calculations *interactively* with *Mathematica*.

Languages like FORTRAN are *compiled*. When you use FORTRAN, you always have to create a complete program. Before you run the program, you first have to “compile” it into machine language. It is effectively impossible to use compiled languages interactively. The traditional advantage of compiled languages has been that they can potentially produce highly efficient code that runs directly on your computer hardware, without the need for an interpreter.

There are two points to realize about interpreted languages like *Mathematica*. First, when you need to solve a problem, there are two parts that take time: formulating the problem on the computer, and actually running the program. For all but the very largest problems, today’s computers make the time you spend running your program negligible compared to the time you spend writing the program. As a result, it will take you less time altogether if you use a system that makes formulating your problem as easy as possible, even if it takes longer to run the program in the end. This is one reason that interpreted languages have become very popular.

Even though simple operations may be slower in an interpreted language than a compiled one, programs you write in *Mathematica* may nevertheless end up being faster than those you write in compiled languages. The main reason is that in most problems it is primarily the algorithm you use, rather than the speed of each step, which determines how fast your whole program will run. When you write a program in *Mathematica*, you automatically get the benefit of the sophisticated algorithms that *Mathematica* contains. The speed of these algorithms often more than makes up for the overhead associated with the interpreter aspects of *Mathematica*.

In some respects, *Mathematica* actually also acts as a compiler. When you enter a program, *Mathematica* converts it to an internal form, which can more efficiently be executed. When you set up transformation rules, *Mathematica* “compiles” them into a specially encoded form, which allows them to be tested and used as quickly

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

as possible.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.